

FounderLink Project Flow and Code Interview Questions

Generated: May 11, 2026

This PDF is prepared for interview revision. It explains the project flow and lists code-based questions an interviewer can ask from this FounderLink codebase.

Project Flow Summary

FounderLink is a full-stack microservices platform that connects startup founders, investors, co-founders, and admins. The frontend is Angular. The backend is split into Spring Boot microservices. Docker Compose runs the supporting infrastructure.

Browser -> Angular frontend -> API Gateway -> Spring Boot microservice -> Controller -> Service -> Repository / Feign / RabbitMQ -> PostgreSQL / Redis / Notification Service -> Response back to Angular

- Angular frontend runs from sprint frontend/founderlink and manages session, forms, dashboards, and service pages through AppStore.
- API Gateway is the single backend entry point on port 8080.
- Config Server centralizes service configuration on port 8888.
- Eureka Server provides service discovery on port 8761.
- PostgreSQL stores auth, user, startup, investment, team, and messaging data.
- Redis is used by user-service for user profile caching.
- RabbitMQ carries notification events to notification-service.

Main User Journey

- A user opens the Angular app and chooses a role such as founder, investor, co-founder, or admin.
- Registration goes to POST /auth/register through the gateway.
- Auth service encodes the password, assigns a role, creates an OTP, syncs profile data to user-service, and publishes an OTP notification event.
- User verifies OTP using POST /auth/verify-otp.
- Login goes to POST /auth/login and returns a JWT.
- Angular stores the JWT and sends it in the Authorization header for protected requests.
- Founders create startup listings through startup-service.
- Admins approve or reject startup listings.
- Investors discover approved startups and create investment requests.
- Founders review investment requests and update status.
- Founders invite co-founders using team-service.
- Users create conversations and send messages through messaging-service.
- Notification-service consumes RabbitMQ events and sends emails.

Service Responsibilities

API Gateway

- Routes frontend calls to /auth, /users, /startups, /investments, /teams, /messages, and /notifications.
- Handles CORS and exposes aggregated Swagger links.

Auth Service

- Owns registration, OTP verification, login, roles, JWT creation, and password reset.

- Uses PasswordEncoder, JWT, PostgreSQL, Feign user profile sync, and RabbitMQ notification events.

User Service

- Owns profile data, directory data, internal user summaries, and profile sync from auth-service.
- Uses Redis cache with @Cacheable and @CachePut.

Startup Service

- Owns startup creation, listing, search, update, admin review, and status notifications.
- Gets authenticated founder id from SecurityContextHolder instead of trusting client-sent userId.

Investment Service

- Owns investment requests and status transitions such as PENDING, APPROVED, REJECTED, and COMPLETED.
- Calls startup-service to find the founder and user-service to find founder email before sending a notification.

Team Service

- Owns co-founder invitations and accepted team memberships.
- Checks that a user can accept only their own invite unless the requester is admin.

Messaging Service

- Owns conversations and messages.
- Reuses existing conversations between two users and checks sender identity before saving messages.

Notification Service

- Consumes RabbitMQ notification.queue events and sends email.
- Uses AmqpRejectAndDontRequeueException to avoid endless retries when email delivery fails.

End-to-End Example Flows

Registration Flow

Angular register form -> POST /auth/register -> AuthController -> AuthService.register -> UserRepository + RoleRepository -> Feign UserProfileClient -> RabbitMQ notification -> OTP email

Create Startup Flow

Founder form -> POST /startups -> JwtFilter -> StartupController -> StartupService.create -> PostgreSQL save -> Feign user lookup -> RabbitMQ notification -> Angular state update

Create Investment Flow

Investor clicks Invest -> POST /investments -> InvestmentService.create -> save PENDING investment -> Feign startup lookup -> Feign user lookup -> RabbitMQ notification -> founder email

Send Message Flow

User sends message -> POST /messages -> MessagingService.sendMessage -> validate sender -> save message -> Feign user summaries -> RabbitMQ notification -> recipient email

Architecture Questions

Q. Why did you choose microservices for FounderLink?

A. Each domain has a separate responsibility: auth, profiles, startups, investments, teams, messaging, and notifications. This improves separation of concerns and makes each service easier to test and scale independently.

Q. What is the role of API Gateway?

A. The gateway gives the frontend one backend URL and routes requests to the correct service. It also centralizes CORS and Swagger aggregation.

Q. Why are Config Server and Eureka used?

A. Config Server centralizes service properties. Eureka allows services to register and discover each other, which is useful when services move, restart, or scale.

Q. Why use RabbitMQ for notifications?

A. Email sending should not block user actions. Services publish NotificationEvent messages, and notification-service handles email delivery asynchronously.

Q. Why is Redis used only in user-service?

A. User profile reads are common across services. Redis reduces repeated database hits for profile lookup while keeping the source of truth in PostgreSQL.

Spring Boot and Java Questions

Q. Explain the controller-service-repository pattern in this project.

A. Controllers expose REST endpoints, services contain business logic, and repositories handle persistence through Spring Data JPA.

Q. What does @RequiredArgsConstructor do here?

A. Lombok generates a constructor for final fields, so Spring can inject dependencies without manual constructor code.

Q. Where is business logic placed?

A. Business logic is mainly in service classes such as AuthService, StartupService, InvestmentService, TeamService, MessagingService, and UserService.

Q. Why is PasswordEncoder used in AuthService?

A. Passwords must not be stored as plain text. PasswordEncoder stores a secure hash and checks login using matches().

Q. How does AuthService normalize roles?

A. It trims input, uppercases it, removes ROLE_ prefix and separators, maps COFUNDER to COFOUNDER, and checks against role records.

Q. What happens if a user logs in without OTP verification?

A. AuthService generates a new OTP, saves it, sends a notification, and rejects login until verification is complete.

Q. Why does StartupService.create set startup id to null?

A. It prevents a detached-entity merge when the client accidentally sends an id during create.

Q. Why does StartupService use SecurityContextHolder?

A. It reads the authenticated user id from the JWT context, so the founder id cannot be forged in the request body.

Q. How is investment status handled?

A. InvestmentService creates new investments as PENDING and updateStatus converts provided status to uppercase before saving.

Q. Why does InvestmentService catch RuntimeException in notifyFounder?

A. Investment creation should still succeed even if startup lookup, user lookup, RabbitMQ, or email infrastructure is temporarily unavailable.

Security Questions

Q. How is JWT used in this project?

A. Auth service generates JWTs after login. Other services use JWT filters to validate the Authorization header and set the authenticated user in the security context.

Q. Why is user id stored as JWT subject?

A. Services can identify the logged-in user without calling auth-service for every request.

Q. How does TeamService prevent accepting someone else's invitation?

A. It compares the authenticated user id with request.userId and allows mismatch only for ROLE_ADMIN.

Q. How does MessagingService protect conversations?

A. It requires the authenticated user to be a participant when creating a conversation and allows users to view only their own conversation list.

Q. What is the risk if services trusted userId from frontend?

A. A malicious client could create startups, investments, messages, or team actions on behalf of another user.

Database and JPA Questions

Q. Why use repositories instead of direct SQL in most places?

A. Spring Data JPA repositories reduce boilerplate and provide common CRUD operations with readable method names.

Q. Where is a custom database operation used?

A. UserRepository has an upsert-style profile sync used by auth-service registration and login recovery.

Q. Why validate email uniqueness in UserService?

A. It prevents two profiles from sharing the same email and allows the current user to keep their own email during update.

Q. What data belongs in auth-service and what belongs in user-service?

A. Auth owns credentials, roles, OTP, and JWT. User-service owns profile fields such as name, bio, skills, experience, links, and location.

Feign and Service Communication Questions

Q. Where is Feign used?

A. Auth syncs profiles to user-service. Startup, investment, team, and messaging services fetch user summaries. Investment also calls startup-service.

Q. Why not duplicate founder email in Investment entity?

A. The source of truth for profile and email data is user-service. Investment stores business data and fetches profile summary when needed.

Q. What should happen if Feign call fails during notification?

A. The main operation should usually succeed, and notification failure should be logged or handled asynchronously.

RabbitMQ Questions

Q. What is NotificationEvent?

A. It is a small DTO carrying email and message from producer services to notification-service.

Q. Which services publish notifications?

A. Auth, startup, investment, team, and messaging services publish notification events.

Q. What does @RabbitListener do?

A. It subscribes notification-service to the configured queue and invokes the consumer method when events arrive.

Q. Why reject without requeue on email failure?

A. If credentials or email data are invalid, requeueing forever can create repeated failures and block the queue.

Angular Questions

Q. What is AppStore responsible for?

A. It centralizes frontend state, session token, users, startups, investments, invites, messages, notifications, and HTTP methods.

Q. How does Angular decide the backend URL?

A. resolveApiBase builds http://current-host:8080, so the frontend calls the gateway on the same host.

Q. Why store the JWT in frontend state/session?

A. The JWT must be added to Authorization headers for protected backend operations.

Q. What is the advantage of mapping backend DTOs to frontend models?

A. The UI can use clean names and types while the backend can keep its own entity and DTO shape.

Testing and Quality Questions

Q. What kinds of tests exist in the project?

A. There are controller tests, service tests, repository tests, JWT filter tests, and application context tests across services.

Q. What is the purpose of SonarQube here?

A. SonarQube reports code quality, coverage, duplication, bugs, vulnerabilities, and maintainability issues.

Q. Why use JaCoCo?

A. JaCoCo generates Java test coverage reports that SonarQube can analyze.

Q. What would you test in AuthService?

A. Registration, duplicate email behavior, OTP expiry, invalid OTP, login before verification, wrong password, role mismatch, JWT generation, and password reset.

Q. What would you test in MessagingService?

A. Conversation reuse, sender validation, participant validation, message ordering, and notification publishing behavior.

Common Interview Follow-ups

Q. Explain one complete flow from frontend click to database save.

A. Example: when a founder creates a startup, the Angular service page calls `AppStore.saveStartup()`. `AppStore` sends `POST /startups` to the API Gateway with the JWT token. The gateway routes the request to `startup-service`. `JwtFilter` validates the token and puts the authenticated user id into `SecurityContextHolder`. `StartupController` receives the request and calls `StartupService.create()`. The service sets id to null, takes founder `userId` from the authenticated context, saves the `Startup` entity through `StartupRepository` into PostgreSQL, optionally publishes a notification event, and returns the saved startup to Angular.

Q. Show where JWT validation happens and how the authenticated user id is used.

A. JWT validation happens in each protected service through its `JwtFilter` class. The filter reads the `Authorization` header, validates the token using `JwtUtil`, extracts the subject, and sets `Authentication` in `SecurityContextHolder`. Services such as `StartupService`, `InvestmentService`, `TeamService`, and `MessagingService` read `SecurityContextHolder.getContext().getAuthentication().getName()` to get the logged-in user id. This prevents clients from spoofing user ids in request bodies.

Q. Explain why notification is asynchronous.

A. Notification is asynchronous because email delivery can be slow or fail due to SMTP or network issues. In `FounderLink`, business services publish `NotificationEvent` messages to `RabbitMQ` and immediately continue their main operation. `notification-service` consumes the event later and sends email. This keeps registration, startup creation, investment creation, team invite, and messaging flows fast and resilient.

Q. Explain how services communicate with each other.

A. The frontend communicates only with API Gateway. The gateway routes HTTP requests to backend services. Backend services communicate synchronously using `OpenFeign` when they need immediate data, such as user-service profile summaries or startup-service startup details. They communicate asynchronously using `RabbitMQ` for notification events. Services also register with `Eureka` and read shared configuration from `Config Server`.

Q. Explain what you would improve if you had more time.

A. I would improve security, resilience, configuration, and testing. For security, I would add refresh tokens and stronger method-level role checks. For resilience, I would add circuit breakers around `Feign` calls and dead-letter queues for failed `RabbitMQ` messages. For maintainability, I would add a centralized error response format and use `Docker secrets` instead of plain environment passwords. For architecture, I would consider separate databases per service. For quality, I would add `Testcontainers`-based integration tests for PostgreSQL, `RabbitMQ`, `Redis`, and service interactions.

Short Improvements List

- Refresh tokens for safer long-running sessions.
- Stronger role-based authorization using method-level checks like `@PreAuthorize`.
- Centralized error response format for consistent frontend handling.
- Circuit breakers and retries for `Feign` service-to-service calls.
- `Docker secrets` or environment vaulting instead of plain passwords.
- Separate databases per service to match true microservice ownership.
- `RabbitMQ` dead-letter queues for failed notification messages.
- Integration tests with `Testcontainers` for database, cache, queue, and service communication.